Doubly Linked List:

1. Insertion at Beginning:

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning(){
    struct node *first,*second,*third,*newnode;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    newnode = (struct node*)malloc(sizeof(struct node));
    first->data = 220;
    second->data = 300;
    third->data = 440;
    newnode->data = 100;
    head = first;
    first->prev = NULL;
    first->next = second;
    second->prev = first;
    second->next = third;
```

Output

```
List: 100 220 300 440

=== Code Execution Successful ===
```

main.c

```c
    first->next = second;
    second->prev = first;
    second->next = third;
    third->prev = second;
    third->next = NULL;
    newnode->prev = NULL;
    newnode->next = head;
    head->prev = newnode;
    head = newnode;
}
void display(){
    struct node *temp = head;
    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
int main(){
    beginning();
    printf("List: ");
    display();
    return 0;
}
```

Output

```
List: 100 220 300 440

=== Code Execution Successful ===
```

2.Insertion at End:

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning(){
    struct node *first,*second,*third;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 20;
    second->data = 40;
    third->data = 60;
    head = first;
    first->prev = NULL;
    first->next = second;
    second->prev = first;
    second->next = third;
    third->prev = second;
```

Output

```
List: 20 40 60 90

=== Code Execution Successful ===
```

main.c

```c
    third->next = NULL;
}
void insertend(){
    struct node *temp,*newnode;
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = 90;
    temp = head;
    while(temp->next != NULL){
        temp = temp->next;
    }
    temp->next = newnode;
    newnode->prev = temp;
    newnode->next = NULL;
}
void display(){
    struct node *temp = head;
    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
int main(){
    beginning();
```

Output

```
List: 20 40 60 90

=== Code Execution Successful ===
```

```c
int main(){
    beginning();
    insertend();
    printf("List: ");
    display();
    return 0;
}
```

3. Insertion at Position:

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning(){
    struct node *first,*second,*third;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    first->data = 10;
    second->data = 30;
    third->data = 50;
    head = first;
    first->prev = NULL;
    first->next = second;
    second->prev = first;
    second->next = third;
    third->prev = second;
    third->next = NULL;
```

Output

```
List: 10 30 35 50

=== Code Execution Successful ===
```

main.c

```c
}
void insertpos(int pos){
    struct node *temp,*newnode;
    int i;
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = 35;
    temp = head;
    for(i=1;i<pos-1;i++){
        temp = temp->next;
    }
    newnode->next = temp->next;
    newnode->prev = temp;
    temp->next->prev = newnode;
    temp->next = newnode;
}
void display(){
    struct node *temp = head;
    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
int main(){
```

Output

```
List: 10 30 35 50

=== Code Execution Successful ===
```

```c
int main(){
    beginning();
    insertpos(3);
    printf("List: ");
    display();
    return 0;
}
```

4. Deletion at Beginning:

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning(){
    struct node *first,*second,*third;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    first->data = 45;
    second->data = 59;
    third->data = 70;
    head = first;
    first->prev = NULL;
    first->next = second;
    second->prev = first;
    second->next = third;
    third->prev = second;
    third->next = NULL;
```

Output

```
List: 59 70

=== Code Execution Successful ===
```

main.c

```c
}
void deletebegin(){
    struct node *temp;
    if(head == NULL){
        printf("List is empty");
        return;
    }
    temp = head;
    head = head->next;
    if(head != NULL){
        head->prev = NULL;
    }
    free(temp);
}
void display(){
    struct node *temp = head;
    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
int main(){
    beginning();
```

Output

```
List: 59 70

=== Code Execution Successful ===
```

```c
int main(){
    beginning();
    deletebegin();
    printf("List: ");
    display();
    return 0;
}
```

5. Deletion at End:

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning(){
    struct node *first,*second,*third;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    first->data = 15;
    second->data = 30;
    third->data = 55;
    head = first;
    first->prev = NULL;
    first->next = second;
    second->prev = first;
    second->next = third;
    third->prev = second;
    third->next = NULL;
```

Output

```
List: 15 30

=== Code Execution Successful ===
```

main.c

```c
}
void deleteend(){
    struct node *temp;
    if(head == NULL){
        printf("List is empty");
        return;
    }
    temp = head;
    while(temp->next != NULL){
        temp = temp->next;
    }
    temp->prev->next = NULL;
    free(temp);
}
void display(){
    struct node *temp = head;
    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
    }
}
int main(){
    beginning();
```

Output

```
List: 15 30

=== Code Execution Successful ===
```

```c
}
int main(){
    beginning();
    deleteend();
    printf("List: ");
    display();
    return 0;
}
```

6. Deletion at Position:

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning(){
    struct node *first,*second,*third;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    first->data = 50;
    second->data = 70;
    third->data = 90;
    head = first;
    first->prev = NULL;
    first->next = second;
    second->prev = first;
    second->next = third;
    third->prev = second;
    third->next = NULL;
```

Output

```
List: 50 90

=== Code Execution Successful ===
```

main.c

Share Run

```c
}
void deletepos(int pos){
    struct node *temp;
    int i;
    if(head == NULL){
        printf("List is empty");
        return;
    }
    temp = head;
    for(i=1;i<pos;i++){
        temp = temp->next;
    }
    temp->prev->next = temp->next;
    if(temp->next != NULL){
        temp->next->prev = temp->prev;
    }
    free(temp);
}
void display(){
    struct node *temp = head;
    while(temp != NULL){
        printf("%d ",temp->data);
        temp = temp->next;
```

Output

```
List: 50 90

=== Code Execution Successful ===
```

```c
        temp = temp->next;
    }
}
int main(){
    beginning();
    deletepos(2);
    printf("List: ");
    display();
    return 0;
}
```